

A Lightweight Modular System for Interactive Mesh Editing Using an Extended Half-Edge

Samuel Rojko
Faculty of Management Science and Informatics
University of Žilina
Žilina, Slovakia
rojko@stud.uniza.sk

Michal Varga
Faculty of Management Science and Informatics
University of Žilina
Žilina, Slovakia
michal.varga@fri.uniza.sk

Abstract—This paper presents a lightweight system for interactive manipulation of meshes. The system is designed with simplicity and modularity in mind, providing an efficient tool for geometric editing. Its architecture enables intuitive, real-time editing of meshes while keeping geometric and rendering data separated. The system emphasizes clarity, extensibility, and maintainability, allowing new tools or operations to be integrated without affecting existing components. The system's internal data structure performance is compared with Blender's BMesh structure, highlighting the trade-offs between implementation simplicity and computational efficiency. For the demonstration, a subsystem to manipulate terrain meshes obtained from OpenTopography is presented.

Keywords—Extended Half-Edge, BMesh, Octree, Terrain Modeling

I. INTRODUCTION

Three-dimensional terrain modeling plays a key role in various domains of computer graphics, simulation, and computer-aided design [1], [2]. Accurate and editable terrain surfaces are essential for applications such as virtual environments, geographic information systems, and interactive design tools [1], [3].

Modern systems like Blender or Fusion 360 provide highly optimized environments for complex geometric editing [4], [5]. However, their extensive toolsets and large-scale architectures make them less suitable for lightweight applications, where simplicity and modularity are prioritized [4], [6], [7]. For research or educational purposes, a smaller, more transparent system can often provide greater flexibility [7], [8], [9].

The paper proposes a lightweight modular system for mesh editing [10]. The system's architecture allows for straightforward integration of new editing tools and features, making it suitable for future development and experiments. It is focused on the internal representation and management of mesh data, enabling flexible geometric operations while keeping focus on a modular and maintainable design.

The paper is organized as follows. In section II there is discussed architecture of the proposed system. Section III describes used spatial data structures and their cooperation. In this section, the used data structure is analytically compared with the BMesh structure used in Blender to highlight the trade-offs between clarity of design and computational performance. Section IV demonstrates the simplicity of proposed design on the implementation of the plugin capable to obtain data from the OpenTopography data source and generate terrain meshes that can be further interactively manipulated via other plugins of the system. Section V summarizes the paper and outlines future work.

II. SYSTEM OVERVIEW

The system is composed of several submodules (as depicted in Fig. 1) that separate *user interaction*, *command execution*, *scene management*, and *rendering*. Each submodule operates independently but communicates through well-defined interfaces, allowing new plugins and functionalities to be integrated with ease.

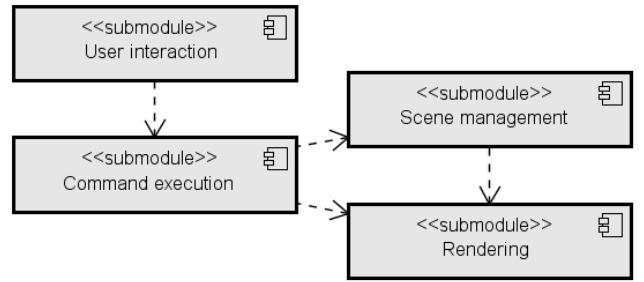


Fig. 1. Submodules of system

A. User interaction submodule

At the top level there lies the User interaction submodule (see Fig. 2 for illustration), which manages all application layers (windows) and user interactions.

Each layer can be registered into the subsystem through LayerRegistry, which maintains and updates all active layers. Every layer provides one or more functionalities accessible through user interaction — for instance, clicking a button, adjusting a parameter, or selecting a tool within the interface.

This design enables clear separation between user interaction and underlying application logic.

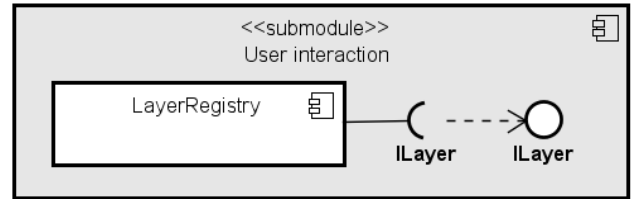


Fig. 2. User interaction submodule

B. Command execution submodule

To execute actions, the Command execution subsystem (see Fig. 3 for illustration) employs a centralized CommandRegistry, which stores and manages all available commands (via general ICommand). These commands are typically bound to some UI components in User interaction submodule.

Each command encapsulates a specific operation and is implemented following the Observer [11] pattern. Every command has an associated callback responsible for the actual implementation of details.

When a command is invoked from the corresponding component, the registry triggers its callback. This approach simplifies extensibility, since new commands can be registered without modifying existing logic.

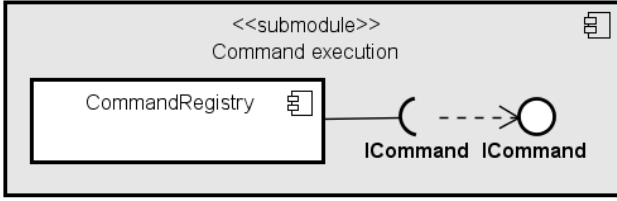


Fig. 3. Command execution submodule

C. Scene management submodule

The Scene acts as a central container responsible for managing all active Meshes and their spatial organization. Scene maintains a collection of octree [12] instances, each representing a region of space that stores a pair of Mesh and (extended) Face references for efficient lookup and updates. When a new Face is created, its bounding box determines which Octree nodes it intersects, ensuring that only relevant regions are updated. Similarly, when a Face is deleted, the Scene removes its references from the corresponding Octree nodes and clears any empty regions, maintaining a compact spatial hierarchy. The diagram of the Scene management subsystem is depicted in Fig. 4.

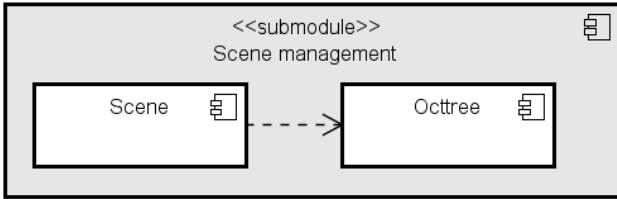


Fig. 4. Scene management subsystem

D. Rendering submodule

Rendering is handled by a dedicated submodule (see Fig. 5 for illustration). It is based on OpenGL. Each Mesh maintains its own buffer layout that defines how vertex data are stored and transmitted to the GPU.

To store only topological and geometric data of the Mesh, without any rendering dependencies, the extended half-edge structure is used as custom implementation of half-edge structure [13]. This ensures a clean separation of concerns and easier maintenance.

During the rendering phase, the system iterates through all visible Meshes and submits their geometry for drawing. Both `drawTriangles()` and `drawLines()` functions are used to visualize the Mesh surfaces and their wireframe outlines.

Each Material is associated with a shader program that controls color and lighting. Shaders are managed through the Shader class, which provides a lightweight abstraction for loading, compiling, and linking OpenGL shaders. This design keeps rendering and geometry management fully decoupled.

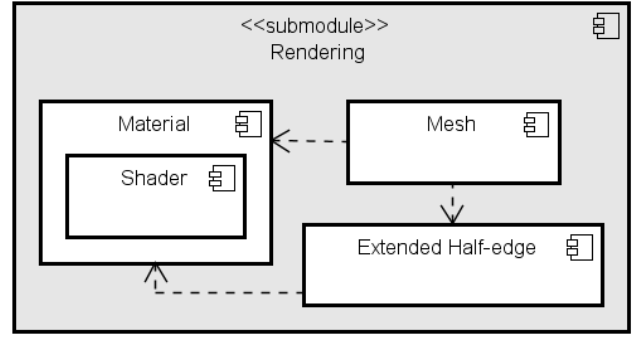


Fig. 5. Rendering subsystem

III. SPATIAL DATA STRUCTURES UTILIZATION

As mentioned in II.C and II.D, two data structures are crucial for proper geometry handling – octree and extended half-edge. The usage of these structures will be discussed in this chapter.

A. Octree

The system uses an octree as its primary spatial partitioning structure for organizing and accessing geometric data within the scene.

The octree recursively subdivides three-dimensional space into smaller cubic regions, where each node corresponds to a specific spatial volume. This hierarchical layout allows for efficient queries, such as ray intersections or region selections without the need to traverse the entire dataset.

In this system, every octree leaf node holds references to Mesh – Face pairs. When a Face is inserted, its bounding box is used to determine all intersecting octree nodes, ensuring that only relevant regions are updated. Since the current implementation supports triangular faces only, the bounding box is computed as an axis-aligned bounding box enclosing the three vertices of the face. The bounding volume is updated whenever vertex positions change due to deformation operations.

The design of octree also allows dynamic updates — nodes can be removed when they become empty, maintaining a compact and memory-efficient hierarchy.

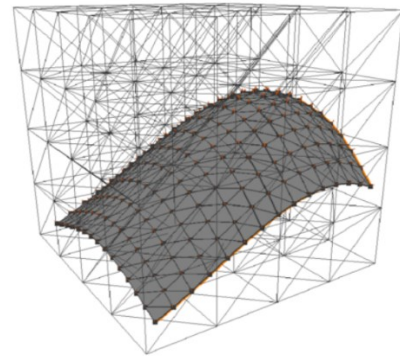


Fig. 6. Visualisation of an octree data structure [14]

B. Half-Edge

At the core of the system lies an extended half-edge structure (see Fig. 6 for illustration) — a topological representation that builds upon the traditional half-edge

structure by introducing additional properties and relationships to support interactive editing. The half-edge structure consists of four interconnected entities:

- **Vertex** — stores the 3D position and a reference to one of its outgoing half-edges.
- **Half-Edge** — represents a directed connection between two vertices and holds references to its twin, next, and previous half-edge, as well as its associated face and edge.
- **Edge** — connects two vertices and links to one of its corresponding half-edges.
- **Face** — maintains a reference to one of its surrounding half-edges.

1) BMesh

Blender’s BMesh [14] (for illustration see Fig. 7) is a dynamic mesh representation designed for real-time editing and high-performance manipulation of complex polygonal geometry. It extends the classic winged-edge [15] structure and supports n-gons, enabling arbitrary face sizes beyond triangles or quads.

Internally, BMesh maintains a set of interconnected elements — BMVert, BMEdge, BMFace, and BMLoop — that collectively describe topological relationships. The BMLoop entity serves a role similar to a half-edge, connecting a Vertex, an Edge and a Face. Each element contains pointers to its related structures, which are stored in memory pools managed by Blender’s allocator. This memory pooling mechanism minimizes allocation overhead and allows rapid creation and deletion of mesh components during interactive operations.

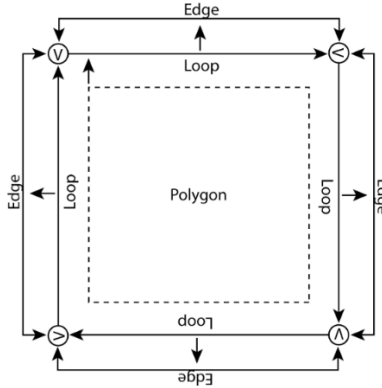


Fig. 7. Illustration of a BMesh structure [14]

2) Extended half-edge

The term extended half-edge in this work does not denote a fundamentally new topological structure, but rather an extension of the standard half-edge representation.

This design choice allows additional editing tools or plugins to be implemented without extending or complicating the core half-edge data structure, which enhances the modularity of the system. The extended half-edge structure (for illustration see Fig. 8) wraps the respective entity with additional relationships and attributes designed to simplify editing:

- **ExtendedVertex** — augments the standard vertex representation by explicitly maintaining lists of

neighboring edges and adjacent faces. While such information can be derived in traditional half-edge structures, storing it directly simplifies and accelerates frequent local editing operations.

- **ExtendedFace** — extends the standard face representation with non-topological attributes such as material references and selection state. By storing these properties directly on faces, common operations, such as material changes or selection updates, can access all required information without additional lookups.
- **GraphEdge** — in addition to standard half-edge relationships, the extended half-edge structure introduces an internal graph layer implemented through the GraphEdge struct. This layer explicitly connects vertices and faces and is maintained independently of the half-edge connectivity. Each GraphEdge represents a bidirectional association between an ExtendedVertex and an adjacent ExtendedFace. Both vertices and faces store lists of their incident graph edges, allowing direct access to neighboring vertices and incident faces without requiring explicit half-edge traversal. While similar adjacency information can be derived in classical half-edge structure, the proposed design stores this connectivity explicitly to simplify and accelerate frequent local operations. This includes neighborhood queries that are used during brush deformations, where repeated traversal of half-edge cycles would otherwise be required.

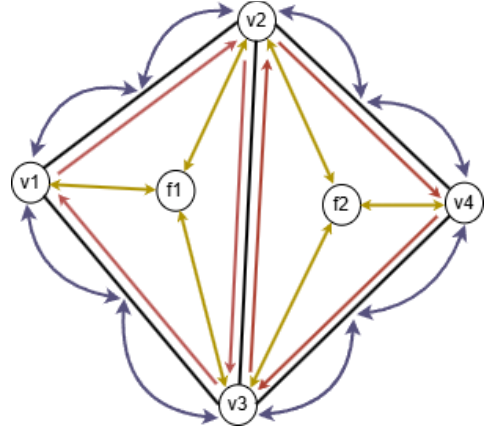


Fig. 8. Illustration of an Extended HalfEdge structure describing two triangles (v1,v2,v3) and (v2,v3,v4)

3) Comparison of BMesh and Extended Half-Edge.

While both BMesh and extended half-edge maintain explicit topological connectivity, several design distinctions exist as summarized in TABLE I.

TABLE I. FEATURE COMPARISONS BETWEEN EXTENDED HALF-EDGE AND BMesh

Feature	Extended half-edge	Blender BMesh
<i>Supported face types</i>	triangular faces only	arbitrary n-gons
<i>Connectivity element</i>	half-edge	BMLoop
<i>Memory management</i>	standard allocation	dynamic memory pooling allocator
<i>Topology traversal</i>	via half-edges	via linked loops

The performance of the structures used was evaluated through two experiments:

- *mesh construction* — The first experiment measured the time required to construct the extended half-edge structure from a predefined set of triangles and compared it against the construction performance of Blender’s BMesh structure. Both representations perform similar initialization steps—allocating vertices, edges, and topological connections—but differ in their underlying memory management strategies.
The extended half-edge implementation relies on standard dynamic allocation with explicit ownership, whereas BMesh employs internal memory pooling that minimizes allocation overhead during large-scale mesh creation.
- *face deletion* — The second experiment measured the time needed to delete all faces within a mesh. This operation reflects how efficiently the data structure can handle element removal and memory cleanup, which are essential for interactive modeling scenarios where geometry is frequently updated.
- *surface deformation using a brush tool* — The third experiment evaluated the performance of an interactive surface deformation operation using a standard brush tool. This test measures the time required to apply a single brush stroke with fixed parameters, affecting a localized region of the mesh. The operation involves identifying affected faces through spatial queries and updating vertex positions accordingly. The same deformation scenario was reproduced in Blender using its built-in draw brush tool for comparison.

These tests were selected because they reflect both the efficiency of the underlying mesh data structure and its performance during interactive editing operations. While the construction and face deletion experiments are parameterized by the number of faces, the deformation experiment is parameterized by the number of affected vertices, as the brush operation directly modifies vertex positions. For simplicity, the evaluation focuses exclusively on the extended half-edge representation implemented in the proposed system and the BMesh structure used internally by Blender, which serves as a well-optimized reference.

All experiments were conducted on a workstation equipped with an AMD Ryzen 5 5600H CPU, 16GB RAM, and an NVIDIA GeForce RTX 3060 Laptop GPU, running Linux Mint. Meshes of varying resolution were generated, with sizes ranging from hundreds to over one million elements. Each configuration was tested in 10 replications to minimize random fluctuations, and the reported values represent the mean execution time in milliseconds.

Results are summarized in TABLE II. and TABLE III. Overall, the evaluation shows that while the implemented extended half-edge structure performs slower than Blender’s BMesh across all experiments, its execution time grows predictably with increasing mesh complexity. The performance gap in the construction and face deletion experiments mainly stems from BMesh’s use of memory pooling strategies, which significantly reduce allocation overhead.

In the surface deformation experiment, the observed performance difference is primarily caused by characteristics of the current implementation rather than inherent limitations of the underlying data structure. The deformation operation is currently executed in a single thread. To ensure consistent and comparable measurements, the reported execution times exclude the cost of redrawing the affected mesh after the deformation. This allows the deformation experiment to focus on the computational cost of the brush operation and vertex position updates.

Despite these limitations, the results demonstrate that the proposed system remains suitable for interactive editing scenarios, especially in contexts where extensibility, architectural clarity, and modularity are prioritized over raw performance.

TABLE II. PERFORMANCE COMPARISON OF MESH CONSTRUCTION

# of faces	Mesh construction (time in ms)		Face deletion (time in ms)	
	Extended Half-Edge	BMesh	Extended Half-Edge	BMesh
45,000	86.8	14.86	47.6	7.99
61,250	146.3	21.2	71.9	12
80,000	158	30.5	83.9	15.74
101,250	208.2	35.45	103.1	18.94
180,000	448.9	70.51	187.8	35.51
245,000	689.6	85.14	272.3	48.93
320,000	834.1	123.69	358.3	64.24
405,000	1164.3	162.09	449.7	85.6
500,000	1462	173.39	572.7	103.71
1,125,000	3075.6	395.75	1021.8	233.93

TABLE III. PERFORMANCE COMPARISON OF MESH DEFORMATION USING A BRUSH TOOL

# of vertices	Surface deformation using a standard brush tool (time in ms)	
	Extended Half-Edge	BMesh
676	62.6	0.21
1,296	87.6	0.33
2,601	128.7	0.57
5,776	229.3	0.71
10,201	371.2	1
22,801	664.6	1.03

C. Integration of octree and extended half-edge structures

The extended half-edge and the octree together form the backbone of the system (as depicted in Fig. 9). While the extended half-edge maintains the topological relationships between faces, edges, and vertices, the octree organizes this data spatially to support efficient queries. Unlike a standard half-edge structure, which focuses solely on topological connectivity, the proposed system explicitly associates each face with a spatial representation in the octree. Each face is referenced by one or more leaf nodes whose spatial volumes intersect with its bounding box, allowing the octree to filter relevant faces during spatial queries without altering the half-edge structure. Whenever a geometric modification occurs — i.e., when a face is added, deformed, or removed — the system first identifies the affected spatial regions. Only the relevant octrees are modified, after which the corresponding extended half-edge elements are updated to maintain topological consistency.

During spatial queries, the octree acts as a high-level filtering layer that rapidly narrows down the set of potentially

affected faces or vertices based on their position in space. Once these regions are identified, detailed geometric and topological operations are performed directly on the corresponding extended half-edge elements. This two-level workflow ensures that editing, selection, and deformation operations remain efficient even for large-scale meshes, since only the localized parts are processed.

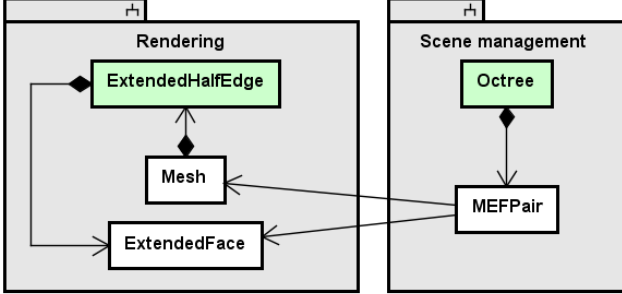


Fig. 9. UML class diagram of spatial data structures

Building upon the geometric and spatial foundations described above, the system provides a set of Mesh editing operations that operate directly on the extended half-edge while utilizing the octree. Currently, these operations are *selection*, *deselection*, *deletion*, and *surface deformation*.

- *Selection* and *deselection* operations are performed through ray–scene intersection tests. When the user clicks on the surface, a ray is cast from the camera's position into the scene. The octree serves as a spatial filter, quickly narrowing down the set of potentially intersected faces. Precise intersection tests are then executed against the extended half-edge data, allowing accurate identification of the selected geometry. Both *selection* and *deselection* operations also update the internal selection states of the corresponding faces or meshes.
- *Deletion* operations remove elements from both the spatial and topological domains. When a face is marked for deletion, the system first determines all octree regions that contain references to this face. These entries are removed from the corresponding octree nodes to maintain a consistent spatial hierarchy. Once the spatial mappings are cleared, the face is deleted from the extended half-edge.
- *Surface deformation* is implemented through the BrushToolPlugin (plugin of proposed system), which performs localized vertex displacement using a standard brush tool. The brush center is obtained from a ray–mesh intersection, and a sphere of radius r defines the region of influence. The octree is queried to identify faces intersecting this region, and vertices inside it are selected for deformation. A single displacement direction is computed from the vertex on the face closest to the brush center, using the average normal of its adjacent faces. All affected vertices are displaced in this direction, with the displacement magnitude determined by their distance from the brush center and the brush strength.

The vertex displacement is defined by the following equations:

$$d_v = \|p_v - c\|$$

$$w_v = \text{smoothstep} \left(\text{clamp} \left(1 - \frac{d_v}{r}, 0, 1 \right) \right)$$

$$\Delta p_v = n \cdot (s \cdot w_v)$$

$$p'_v = p_v + \Delta p_v$$

Here:

- p_v is the position of vertex p ,
- c is the brush center,
- r is the brush radius,
- s is the brush strength,
- n is the displacement direction computed from the closest hit vertex

IV. OPENTOPOGRAPHY DATA FETCHER PLUGIN

We demonstrate straightforward integration of new features on the plugin capable of fetching data from the OpenTopography [16] public API and generate the respective mesh from high-resolution digital elevation models and point cloud datasets provided by OpenTopography.

To define a plugin, one needs only to define respective `ILayer` and `ICommand` and register them into the corresponding registers – see Fig. 10 for component diagram of OpenTopography plugin and Fig. 2 and Fig. 3 for registers.

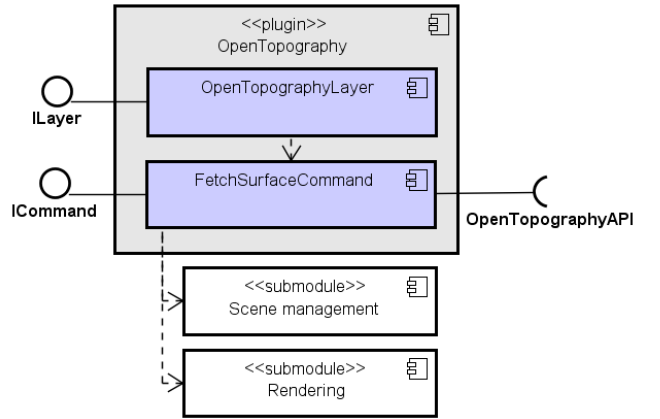


Fig. 10. OpenTopography data fetcher plugin

`FetchSurfaceCommand` queries the API and converts the downloaded elevation data into a mesh representation. Each dataset is retrieved in raster format, where elevation values are sampled at regular grid intervals. These samples are converted into vertices, and triangular faces are formed by connecting neighboring vertices across the grid.

The user interface (`OpenTopographyLayer`) is depicted in Fig. 11.

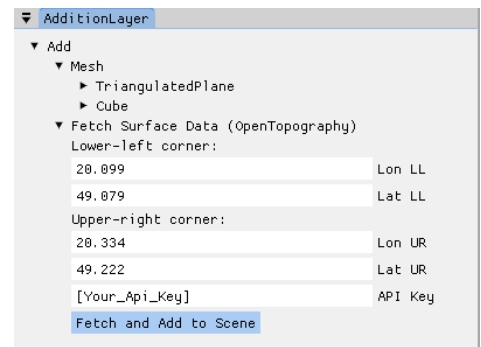


Fig. 11. User interface of OpenTopography data fetcher plugin

Lastly, we demonstrate mesh generated using the proposed plugin – the selected High Tatras region. Generated mesh, as depicted in Fig. 12, contains 865,944 faces.

V. CONCLUSION AND FUTURE WORK

This paper presented the design and implementation of a lightweight, modular system for interactive manipulation of meshes. Its design proves that efficient geometric processing can be achieved even within a lightweight framework focused on clarity and ease of extension. The simplicity of extensions was demonstrated on plugin capable of generating terrain meshes from the data obtained from the OpenTopography.

Although the current implementation prioritizes clarity and modularity over raw performance, it provides a strong

foundation for further experimentation and tools development. The evaluation revealed several optimization opportunities, especially in memory allocation efficiency.

Future work will focus on extending functionality and improving performance. One of the planned enhancements is the introduction of a memory pooling system to reduce allocation overhead during mesh creation and deletion. Support for undo and redo operations would be added to make editing more flexible.

The system would also include scene serialization and deserialization, allowing entire scenes to be saved and reloaded. In addition, several new tools are planned, targeting both hard-surface modeling and organic shape editing, further expanding the system's capabilities.

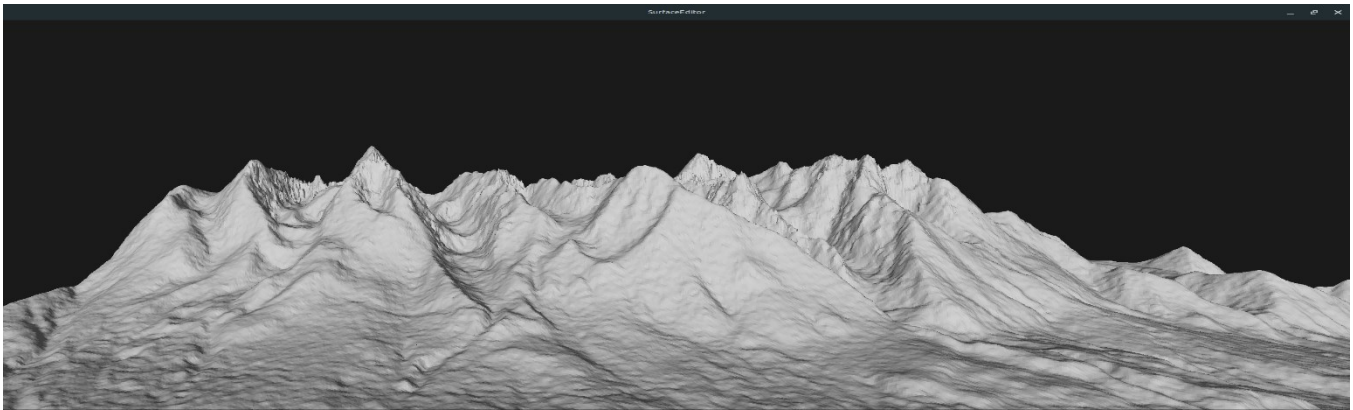


Fig. 12. 3D terrain visualization of the High Tatras region processed by OpenTopography data fetcher plugin

References

- [1] E. Galin, E. Guérin, A. Peytavie, G. Cordonnier, M.-P. Cani, B. Benes a J. Gain, „A Review of Digital Terrain Modeling,” *Computer Graphics Forum*, zv. 38, %1. vyd.2, p. 553–577, 2019.
- [2] R. M. Smelik, T. Tutenel, R. Bidarra a B. Benes, „A Survey on Procedural Modelling for Virtual Worlds,” *Computer Graphics Forum*, zv. 33, %1. vyd.6, p. 31–50, 2014.
- [3] C. M. C. M. Ruzinoor, A. R. M. Shariff, B. Pradhan, M. R. Ahmad a M. S. M. Rahim, „A review on 3D terrain visualization of GIS data: techniques and software,” *Geo-spatial Information Science*, zv. 15, %1. vyd.2, pp. 105-115, 2012.
- [4] Blender Foundation, „Blender — The Free and Open Source 3D Creation Suite,” 10 2025. [Online]. Available: <https://www.blender.org/>. [Cit. 10 2025].
- [5] Autodesk, „Fusion 360 — 3D CAD, CAM, CAE & PCB,” 10 2025. [Online]. Available: <https://www.autodesk.com/products/fusion-360/overview>. [Cit. 10 2025].
- [6] R. M. Smelik, K. J. d. Kraker, S. A. Groenewegen, T. Tutenel a R. Bidarra, „A Survey of Procedural Methods for Terrain Modelling,” rev. *Proceedings of the CASA Workshop on 3D Advanced Media in Gaming and Simulation (3AMIGAS)*, 2009.
- [7] OpenSCAD Project, „OpenSCAD — The Programmers Solid 3D CAD Modeller,” 10 2025. [Online]. Available: <https://openscad.org/>. [Cit. 10 2025].
- [8] Visual Computing Lab (CNR-ISTI), „MeshLab — an Open Source System for Processing and Editing 3D Triangular,” 10 2025. [Online]. Available: <https://www.meshlab.net/>. [Cit. 10 2025].
- [9] three.js contributors, „three.js — JavaScript 3D Library,” 10 2025. [Online]. Available: <https://threejs.org/>. [Cit. 10 2025].
- [10] iomko, „SurfaceEditor — application that allows us to manipulate with planar or non-planar surfaces,” 2025. [Online]. Available: <https://github.com/iomko/SurfaceEditor>. [Cit. 10 2025].
- [11] E. Gamma, R. Helm, R. Johnson a J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995, p. 293–303.
- [12] H. Samet, „The Quadtree and Related Hierarchical Data Structures,” *ACM Computing Surveys*, zv. 16, %1. vyd.2, p. 187–260, 1984.
- [13] L. Kettner, „Using Generic Programming for Designing a Data Structure for Polyhedral Surfaces,” rev. *Proceedings of the 14th Annual Symposium on Computational Geometry (SoCG'98)*, 1998.
- [14] S. Rojko, „Application for surface modification in three-dimensional space,” 2024.
- [15] Blender Foundation, „BMesh — Blender Developer Documentation“.
- [16] B. G. Baumgart, „A Polyhedron Representation for Computer Vision,” rev. *Proceedings of the National Computer Conference (AFIPS '75)*, 1975.
- [17] OpenTopography, „OpenTopography — High-Resolution Topography Data and Tools,” 2025. [Online]. Available: <https://opentopography.org/>. [Cit. 10 2025].